

CS247 – SPRING 26 – TUTORIAL 4

M7 – ASSERTIONS AND EXCEPTIONS

SMART POINTERS & RAII IDIOM

The Tutorial's Quick Tutorial

1. The topics covered by Prof. Victoria on her slides will be further expanded during our meetings.
2. Each topic will have up to 3 subsections:
 1. A quick recap;
 2. an **Example**;
 3. a **Challenge**;
3. We will use Microsoft Copilot Chat (<https://office.com/chat>), specially with the proposed **Challenges**, to evaluate how AI behaves with these “*gotcha!*” questions, so we can learn to evaluate AI’s strengths and shortcomings.
4. Please, review UW’s “**Responsible Use of AI Tools with University Data**” guidelines at <https://uwaterloo.ca/information-systems-technology/about/policies-standards-and-guidelines/artificial-intelligence-ai/responsible-use-ai-tools-university-data>
5. Please, review the course’s guidelines regarding the usage of AI at https://outline.uwaterloo.ca/viewer/view/nye4ps#generative_ai

Supporting Material

Repo: <https://github.com/talesmp/CS247-S26-Tutorial4.git>

Vevox Session

Join at:
vevox.app

ID:
489-682-454



The Thread

ASSUME → SIGNAL → UNWIND → LEAK → OWN → ENFORCE

ASSUME	SIGNAL	UNWIND	LEAK	OWN	ENFORCE
assert (no unwind)	throw/typed, slicing	handler order, rethrow, ctor	raw ptr leaks; safety	smart pointer + RAII	noexcept + terminate

The Main Question

When this throws, whose destructor runs — and what leaks?

- **assert** / **abort** → *nothing* unwinds (that's the point)
- **exception** → stack objects' destructors run, in reverse
- **half-built object** → its destructor is *skipped*
- **raw new** → leaks · **smart pointer** / **RAII** → freed
- **noexcept violation** → **terminate**, no unwinding

Phase 1 – ASSUME

Document & check your assumptions

ASSUME → SIGNAL → UNWIND → LEAK → OWN → ENFORCE

1. Assertions & the side-effect trap

- `assert` is a **preprocessor macro** → `-DNDEBUG` deletes it
- the `i++` vanishes in release → **flag-dependent behavior**
- **Fix:** `assert(i == 10); ++i;` — thermometer, not thermostat

Phase 2 – SIGNAL

Throw an error up the stack as an object

ASSUME → **SIGNAL** → UNWIND → LEAK → OWN → ENFORCE

2. Throw a typed object

- `catch` selects by **type**, with **no conversions**
- a `const char` *is not a* `std::exception` $\rightarrow$ *no handler* $\rightarrow$ *terminate*
- **Fix:** throw a typed object derived from `std::exception` (carries `what()` + a payload)

3. Catch by const&, not by value

- by value → the `runtime_error` is **sliced** to a base `std::exception`
- `virtual what()` now dispatches to the base
- **Fix:** `catch (const std::exception& e)` — keeps the true type

Phase 3 – UNWIND

How the error travels, and what gets destroyed

ASSUME → SIGNAL → UNWIND → LEAK → OWN → ENFORCE

4. Handler order: derived before base

- first match wins; base IS-A catches all derived → later clause unreachable
- **Which tool? the compiler** — caught before it ran
- **Fix:** specific-first; catch (...) last

5. Rethrow: throw; vs throw e;

- `throw e;` slices to `Base` → outer `catch(Derived&)` misses → terminate
- **Fix:** bare `throw;` re-launches the *same* object, real type intact → inner / outer

6. Construction throws

- a destructor tears down a **fully built** object only
- half-built **c** → **no destructor**; finished **a**, **b** unwind in reverse
- Observation: There's no fix, this is unwinding doing its job

Phase 4 – LEAK

The gap unwinding doesn't cover

ASSUME → SIGNAL → UNWIND → LEAK → OWN → ENFORCE

7. The raw-pointer leak

- unwinding pops the pointer `r`, but a raw pointer has **no destructor**
- the heap object is stranded → leaked
- **Fix:** `auto r = std::make_unique<Resource>();` → freed on every path

8. Exception safety: basic vs strong

- “caught it” $\neq$ “left things consistent” $\rightarrow$ not even **basic**-safe
- **Fix (strong):** on failure undo step 1, then bare `throw;` $\rightarrow$ total stays 100

Phase 5 – OWN

Tie each resource to a stack object

ASSUME → SIGNAL → UNWIND → LEAK → **OWN** → ENFORCE

9. unique_ptr is non-copyable

- exclusive ownership → copy ctor **deleted** (no second owner)
- **Fix:** `auto q = std::move(p);` → `p` null, `q` owns it, `q == 10`
 - returning one needs no `move`; passing one in does

10. shared_ptr reference cycle

- each use count drops $2 \rightarrow 1$, never $0 \rightarrow$ they keep each other alive
- not a `shared_ptr` bug — an **ownership-design** flaw
- **Fix:** make one back-edge a `weak_ptr` $\rightarrow$ end of main / $\sim A$ / $\sim B$

11. RAII – the destructor is the safety net

- acquire in ctor, **release in dtor** → cleanup is exception-safe by construction
- smart pointers are just RAII applied to memory
- Observation: no fix $\Rightarrow$ It's the idiom working

Phase 6 – ENFORCE

Promise no-throw, and terminate enforces it

ASSUME → SIGNAL → UNWIND → LEAK → OWN → ENFORCE

12. noexcept is a hard promise

- throw from `noexcept` → straight to `std::terminate`, no handler search
- **Fix:** if it might throw, **don't** mark it `noexcept`

Conclusion

When it throws, the stack unwinds — so put every resource on the stack.

- **assertions** halt our *bugs* with no unwinding; **exceptions** unwind for the world's problems
- unwinding runs stack destructors automatically — but **not** raw heap pointers
- **smart pointers** + **RAII** put the heap on the stack → leaks become impossible
- `noexcept` + `std::terminate` enforce the contract

UNIVERSITY OF
WATERLOO



FACULTY OF MATHEMATICS
QUESTIONS?

THANK YOU!

tmellopaiva@uwaterloo.ca